Lecture 5. Iterative Methods for Linear Systems

Splitting, spectral radius, Jacobi, Gauss–Seidel, SOR, conjugate gradient

Contents

5.1 Direct versus iterative methods **2**

5.2 Splitting and convergence **2**

5.3 Classical iterations **3**

5.4 Conjugate gradient **4**

5.1 Direct versus iterative methods

Direct methods (LU, Cholesky, QR, SVD) have a cost fixed in advance and are accurate; they suit dense matrices of small or moderate size. **Iterative methods** (Jacobi, Gauss–Seidel, SOR, conjugate gradient, GMRES) produce a sequence of approximations whose accuracy depends on the number of iterations; they suit very large, sparse or structured systems, parallel computing, and warm starts.

Example 5.1 (PageRank)

Let $a_{ij} = 1$ if web page i links to page j, and $r_i = \sum_j a_{ij}$ the out-degree. A random surfer at a page with $r_i > 0$ follows a random link with probability p and jumps to a uniformly random page otherwise; from a page with no links, the surfer jumps uniformly. This Markov chain has transition probabilities

$$p_{ij} = \frac{1-p}{n} + p\frac{a_{ij}}{r_i} \qquad (a_{ij}/r_i := 1/n \text{ if } r_i = 0),$$

and the page scores are its stationary distribution: $x^\top P = x^\top$, i.e. $(I - P^\top)x = 0$. With $n \approx 10^9$ pages, LU would take $\frac{2}{3}(10^9)^3/10^{12}$ seconds, about 20 million years on a teraflop machine. Iterative methods, which only need products with the sparse link matrix, make it feasible. The dense matrix $I - P^\top$ is never formed: row i of $(I - P^\top)x$ is $x_i - \frac{1-p}{n}\sum_{j:r_j>0} x_j - \frac{1}{n}\sum_{j:r_j=0} x_j - p\sum_{j\to i} x_j/r_j$, where the last sum runs over the pages linking to i (a column of the sparse A). The two global sums are kept as running totals, so a full Gauss–Seidel or SOR sweep costs $O(n + \mathrm{nnz}(A))$.

5.2 Splitting and convergence

Split $A = M - K$ with M invertible and easy to invert. Then $Ax = b$ iff $x = M^{-1}Kx + M^{-1}b$, so a solution is a fixed point of

$$x^{(t+1)} = Rx^{(t)} + c, \qquad R = M^{-1}K, \quad c = M^{-1}b.$$

Norms. A vector norm satisfies $\|x\| \geq 0$, $\|x\| = 0 \iff x = 0$, $\|cx\| = |c|\|x\|$ and the triangle inequality; common choices are ℓ_1, ℓ_2, ℓ_∞. A matrix norm is also required to be submultiplicative, $\|AB\| \leq \|A\|\|B\|$. Examples: the Frobenius norm $\|A\|_F = \sqrt{\mathrm{tr}(AA^\top)}$, and the **operator norm** $\|A\| = \sup_{y\neq 0}\|Ay\|/\|y\|$ induced by a vector norm, with

$$\|A\|_1 = \max_j \sum_i |a_{ij}|, \qquad \|A\|_\infty = \max_i \sum_j |a_{ij}|, \qquad \|A\|_2 = \sqrt{\rho(A^\top A)},$$

where $\rho(B) = \max_i|\lambda_i(B)|$ is the **spectral radius**.

If $x^\star$ solves $Ax = b$, then $x^{(t+1)} - x^\star = R(x^{(t)} - x^\star)$, so $\|x^{(t+1)} - x^\star\| \leq \|R\|\|x^{(t)} - x^\star\|$, and $\|R\| < 1$ for some operator norm implies convergence.

Theorem 5.1

For every operator norm, $\rho(A) \leq \|A\|$. Conversely, for every $\epsilon > 0$ there is an operator norm with $\|A\| \leq \rho(A) + \epsilon$. Hence $\rho(A) = \inf\|A\|$ over operator norms, and $\rho(A) < 1$ iff $\|A\| < 1$ for some operator norm.

Proof. If $Av = \lambda v$ with $v \neq 0$ (complex in general), then $|\lambda|\|v\| = \|Av\| \leq \|A\|\|v\|$.

For the converse, use the Schur decomposition $A = STS^{-1}$ with T upper triangular whose diagonal holds the eigenvalues. For $\delta > 0$ let $D_\delta = \operatorname{diag}(1, \delta, \dots, \delta^{n-1})$. Then $T_\delta = D_\delta^{-1} T D_\delta = (SD_\delta)^{-1} A (SD_\delta)$ has entries $\delta^{j-i} t_{ij}$ $(j \geq i)$: the off-diagonal part shrinks as $\delta \to 0$. Define $\|x\|_\delta = \|(SD_\delta)^{-1} x\|_\infty$; it is a vector norm because $M = (SD_\delta)^{-1}$ is invertible (nonnegativity, homogeneity and the triangle inequality transfer from $\|\cdot\|_\infty$, and $\|Mx\|_\infty = 0 \iff Mx = 0 \iff x = 0$). Substituting $y = (SD_\delta)^{-1} x$, its induced norm is

$$\|A\|_\delta = \sup_{y \neq 0} \frac{\|(SD_\delta)^{-1} A (SD_\delta) y\|_\infty}{\|y\|_\infty} = \|T_\delta\|_\infty = \max_i \sum_{j \geq i} |t_{ij}| \delta^{j-i},$$

which is at most $\max_i |t_{ii}| + \epsilon = \rho(A) + \epsilon$ for small δ. ■

Corollary 5.2

If A is invertible, the iteration $x^{(t+1)} = Rx^{(t)} + c$ converges to $A^{-1}b$ for every $x^{(0)}$ if and only if $\rho(R) < 1$.

Proof. If $\rho(R) < 1$, pick an operator norm with $\|R\| < 1$ by Theorem 5.1; then $\|x^{(t)} - x^\star\| \leq \|R\|^t \|x^{(0)} - x^\star\| \to 0$. (This direction does not need A invertible; the limit is then some solution.) Conversely, if $|\lambda| \geq 1$ for an eigenpair $Rv = \lambda v$, start at $x^{(0)} = x^\star + v$: then $\|x^{(t)} - x^\star\| = |\lambda|^t \|v\| \geq \|v\| \neq 0$, so the iteration does not converge. ■

5.3 Classical iterations

Write $A = L + D + U$ (strictly lower, diagonal, strictly upper parts).

Jacobi. $M = D$, $K = -(L + U)$:

$$x_i^{(t+1)} = \frac{1}{a_{ii}} \Big(b_i - \sum_{j \neq i} a_{ij} x_j^{(t)} \Big), \qquad x^{(t+1)} = x^{(t)} + D^{-1}(b - Ax^{(t)}).$$

Gauss–Seidel. $M = D + L$ (lower triangular, easy to invert by forward substitution), $K = -U$: newly computed components are used immediately,

$$x_i^{(t+1)} = \frac{1}{a_{ii}} \Big(b_i - \sum_{j<i} a_{ij} x_j^{(t+1)} - \sum_{j>i} a_{ij} x_j^{(t)} \Big).$$

It is the linear-system analogue of coordinate descent. In code the two methods differ by one detail: a loop over i that writes the new x_i back into x before computing the next component is Gauss–Seidel, while one that stores all new components separately and replaces x only after the sweep is Jacobi. Gauss–Seidel needs no second vector, but its sweep is inherently sequential, whereas Jacobi vectorizes and parallelizes.

Successive over-relaxation (SOR). A weighted combination of the Gauss–Seidel update and the old value:

$$x_i^{(t+1)} = (1 - \omega) x_i^{(t)} + \frac{\omega}{a_{ii}} \Big(b_i - \sum_{j<i} a_{ij} x_j^{(t+1)} - \sum_{j>i} a_{ij} x_j^{(t)} \Big),$$

i.e. $M = \frac{1}{\omega} D + L$, $K = \frac{1-\omega}{\omega} D - U$. $\omega = 1$ is Gauss–Seidel; $\omega > 1$ (over-relaxation) can accelerate convergence substantially.

Proposition 5.3

If A is strictly diagonally dominant by rows, $|a_{ii}| > \sum_{j\neq i}|a_{ij}|$, then the Jacobi and Gauss–Seidel iterations converge for every starting point.

Proof. *Jacobi.* $R = -D^{-1}(L+U)$ has $\|R\|_\infty = \max_i \sum_{j\neq i}|a_{ij}|/|a_{ii}| < 1$. *Gauss–Seidel.* Let $Rv = \lambda v$ with $R = -(D+L)^{-1}U$, so $-Uv = \lambda(D+L)v$, and choose i with $|v_i| = \|v\|_\infty = 1$. The i-th row gives $\lambda a_{ii}v_i = -\sum_{j>i} a_{ij}v_j - \lambda\sum_{j<i} a_{ij}v_j$, hence

$$|\lambda|\Big(|a_{ii}| - \sum_{j<i}|a_{ij}|\Big) \le \sum_{j>i}|a_{ij}|,$$

and the right side is smaller than the bracket by diagonal dominance. So $|\lambda| < 1$ and $\rho(R) < 1$. ■

Jacobi also converges if A is strictly diagonally dominant by *columns*, $|a_{jj}| > \sum_{i\neq j}|a_{ij}|$: then $A^\top$ is row dominant, and $R = -D^{-1}(L+U)$ is similar to $DRD^{-1} = -(L+U)D^{-1}$, whose $\|\cdot\|_1$ (maximum column sum) is $\max_j \sum_{i\neq j}|a_{ij}|/|a_{jj}| < 1$. Similar matrices have the same spectral radius.

Example 5.2 (Jacobi for the second-difference matrix)

Let $A = \mathrm{tridiag}(-1, 2, -1) \in \mathbb{R}^{n\times n}$, whose eigenvalues are $\lambda_i = 2 - 2\cos\frac{i\pi}{n+1}$ with orthonormal eigenvectors v_i. Jacobi has $D = 2I$ and $R = -D^{-1}(L+U) = I - \frac{1}{2}A$, so R has the same eigenvectors and eigenvalues

$$\mu_i = 1 - \tfrac{1}{2}\lambda_i = \cos\frac{i\pi}{n+1}, \qquad \rho(R) = \cos\frac{\pi}{n+1} < 1.$$

The method converges, but slowly: $\rho(R) \approx 1 - \frac{\pi^2}{2(n+1)^2}$, so reducing the error by a constant factor takes $O(n^2)$ sweeps. Writing the error as $\varepsilon^{(k)} = \sum_i a_i^{(k)} v_i$, the recursion $\varepsilon^{(k+1)} = R\varepsilon^{(k)}$ gives $a_i^{(k)} = \mu_i^k a_i^{(0)}$: each mode decays geometrically at its own rate $|\mu_i|$. Modes with i near $(n+1)/2$ ($\mu_i \approx 0$) die almost at once, while the smoothest mode $i = 1$ and the most oscillatory mode $i = n$ ($\mu_n = -\mu_1$, sign-alternating) persist. This observation is the starting point of multigrid methods, which treat smooth error components on coarser grids.

Cost. One sweep costs $2n^2$ flops for a dense A, so iteration beats LU only if it converges in $o(n)$ sweeps. The real gain is for sparse or structured A, where Av costs $O(n)$ or $O(n\log n)$ and A need not even be stored (e.g. a finite-difference operator $(Av)_i = v_i - v_{i+1}$).

5.4 Conjugate gradient

For symmetric positive definite A, solving $Ax = b$ is equivalent to minimizing the convex quadratic $\frac{1}{2}x^\top Ax - b^\top x$, whose gradient is $Ax - b$. The **conjugate gradient** method minimizes it along mutually A-conjugate directions and, in exact arithmetic, terminates in at most n steps; in practice, especially with preconditioning, it converges in far fewer. It is the method of choice for huge structured positive definite systems. (In a classical plasma physics example, Gauss–Seidel needed 208,000 iterations, block SOR 765, and incomplete-Cholesky-preconditioned CG 25.) The details belong to an advanced course.

A numerical test on the Poisson matrix (a sparse, symmetric positive definite block tridiagonal matrix from the discretized Laplacian) shows the pattern: Jacobi needs thousands of iterations, Gauss–Seidel and SOR fewer, and CG far fewer, all much cheaper than a dense Cholesky solve. Once again: *exploit structure.*